

## Session 6 – SLO 2

### Imperative Paradigm - Procedural — Modular Programming — Answers

---

Q1. Break a program into multiple reusable methods in Java.

The long processOrder function from the previous question can be broken down into smaller, reusable methods, each handling a single responsibility.

```
public class OrderProcessorModular {
    static boolean isValid(double price, int qty) {
        return price > 0 && qty > 0;
    }

    static double calculateSubtotal(double price, int qty) {
        return price * qty;
    }

    static double applyDiscount(double amount, boolean isMember) {
        return isMember ? amount * 0.9 : amount;
    }

    static double calculateTax(double amount) {
        return amount * 0.05;
    }

    static void printReceipt(String customer, double subtotal,
                             double tax, double finalAmount) {
        System.out.println("Customer: " + customer);
        System.out.println("Subtotal: " + subtotal);
        System.out.println("Tax: " + tax);
        System.out.println("Final Amount: " + finalAmount);
    }

    public static void main(String[] args) {
        String customer = "Anita";
        double price = 250.0;
        int qty = 3;
        boolean isMember = true;

        if (!isValid(price, qty)) {
            System.out.println("Invalid order");
            return;
        }
    }
}
```

```

    }

    double subtotal = applyDiscount(calculateSubtotal(price, qty), isMember);
    double tax = calculateTax(subtotal);
    printReceipt(customer, subtotal, tax, subtotal + tax);
}
}

```

Each concern -- validation, subtotal calculation, discounting, tax calculation, and printing -- now lives in its own small, independently testable and reusable method, making the overall program easier to read and maintain.

Q2. Write a Java program that uses a main method and two helper methods.

```

public class TemperatureConverter {
    static double celsiusToFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    static double fahrenheitToCelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }

    public static void main(String[] args) {
        double c = 25.0;
        double f = 98.6;

        System.out.println(c + "C = " + celsiusToFahrenheit(c) + "F");
        System.out.println(f + "F = " + fahrenheitToCelsius(f) + "C");
    }
}

```

The main method acts as the entry point and coordinator, calling the two helper methods `celsiusToFahrenheit` and `fahrenheitToCelsius`, each of which performs one clearly defined conversion task and can be reused independently wherever temperature conversion is needed.

Q3. Explain how Java supports code reuse via static methods.

Static methods support code reuse in Java in several important ways:

- Callable without an object: a static method belongs to the class itself, so it can be invoked directly using the class name (for example, `Math.sqrt(25)`), without needing to create a new instance every time it is used.

- Shared utility logic: common operations (validation, formatting, mathematical calculations) can be written once as static methods and reused from anywhere in the program, avoiding duplicated code.
- Utility classes: classes such as Java's built-in Math and Collections classes are made up almost entirely of static methods, providing ready-to-use, reusable functionality across any program.
- Consistent behavior: since a static method operates only on the parameters it is given (when written without touching shared mutable state), calling it repeatedly with the same input reliably produces the same output, making it safe and predictable to reuse.

For example, a static method such as `calculateTax(double amount)` can be defined once and reused across many different parts of a large application, ensuring the same tax logic is applied consistently everywhere it is called.